KANT AX/AI 엔지니어 트랙 · 04

Git diff, 눈으로 보기

무엇이 바뀌었는지
커밋 전에 확인해요

```
@@ -1,3 +1,4 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
```

오늘의 순서

1	diff란?	status와 diff, 비교하는 두 칸	3–4쪽
2	`git diff`	add 전에 고친 줄 보기, 출력 읽기	5–8쪽
3	`git diff --staged`	커밋될 내용 확인하기	9–12쪽
4	`git show`와 브랜치 비교	커밋 하나 보기, merge 전에 미리 보기	13–15쪽
5	`.gitignore`	올리면 안 되는 파일 빼기	16–18쪽
6	비교와 막힐 때	어떤 명령을 칠까, 빈 화면, 안 끝나는 화면	19–22쪽
7	실제 상황	충돌 미리 보고 풀기, 빠진 파일 찾기	23–26쪽
8	한 장 요약		27쪽

diff란?

status는 파일 이름만, diff는 바뀐 줄까지 보여 줘요

```
$ git status
Changes not staged for commit:
        modified:   login.md
```

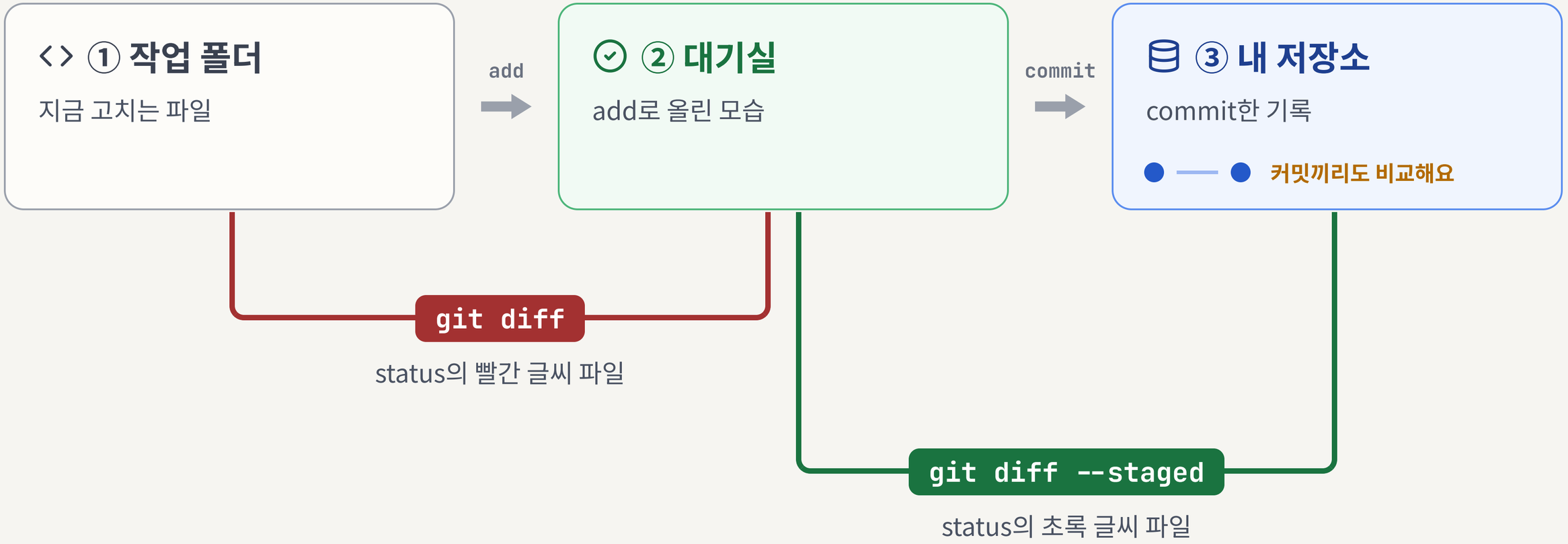
어느 파일이 바뀌었는지만 알려 줘요

```
$ git diff
diff --git a/login.md b/login.md
index ef88ce3..7d67f3e 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,4 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
```

어느 줄이 어떻게 바뀌었는지까지 - 지운 줄, + 넣은 줄

diff란?

diff는 두 칸을 골라 나란히 놓고 비교해요



실습 1 · git diff

git diff

```
$ git diff
diff --git a/login.md b/login.md
index ef88ce3..7d67f3e 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,4 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
```

고친 줄만 나와요

출력이 길어서 아래에 : 가 보이고 멈추면 q를 눌러 나와요.



출력 읽기

- 는 지운 줄, + 는 넣은 줄이에요



한 줄을 고치면 - 한 줄과 + 한 줄이 짝으로 나와요. 예전 줄을 지우고 새 줄을 넣은 걸로 보여 줘요.

출력 읽기

@@ 줄은 고친 곳의 위치를 알려 줘요

```
@@ -1,3 +1,4 @@
```

-1,3 1번째 줄부터 3줄 예전 파일

```
1  # 로그인
2  아이디
3  비밀번호
```

+1,4 1번째 줄부터 4줄 지금 파일

```
1  # 로그인
2  아이디
3  비밀번호 (8자 이상)
4  로그인 버튼
```

숫자는 몰라도 diff를 읽을 수 있어요. 긴 파일에서 고친 곳을 찾아갈 때 써요.

실습 2 · add한 다음 git diff

git diff

```
$ git add login.md
$ git diff
# 아무것도 안 나와요
```

①과 ②가 똑같아서 보여 줄 줄이 없어요.
고친 내용은 ②로 옮겨 갔을 뿐 그대로 있어요.



실습 3 · git diff --staged

git diff --staged

```
$ git diff --staged
diff --git a/login.md b/login.md
index ef88ce3..7d67f3e 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,4 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
```

commit하면 들어갈 줄

커밋 메시지를 쓰기 전에 한 번 보면 무엇을 했는지 바로 적을 수 있어요.



실습 4 · add 뒤에 또 고치면

git diff

```
$ echo "아이디 저장" >> login.md
$ git diff
diff --git a/login.md b/login.md
index 7d67f3e..60a5971 100644
--- a/login.md
+++ b/login.md
@@ -2,3 +2,4 @@
 아이디
 비밀번호 (8자 이상)
 로그인 버튼
+아이디 저장
```

add 뒤에 고친 한 줄만

기준이 대기실이라서 add 뒤에 고친 줄만 보여요.



실습 4 · add 뒤에 또 고치면

git diff --staged

```
$ git diff --staged
diff --git a/login.md b/login.md
index ef88ce3..7d67f3e 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,4 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
```

지금 commit하면 **아이디 저장**은 빠져요. 한 번 더 git add 해요.



실습 5 · 커밋하고 나면

커밋하고 나면 git diff도 --staged도 비어요

```
$ git add login.md
$ git commit -m "feat: 로그인 화면에 안내와 버튼 추가"
[main 7e5303c] feat: 로그인 화면에 안내와 버튼 추가
 1 file changed, 3 insertions(+), 1 deletion(-)
$ git diff
$ git diff --staged
# 둘 다 아무것도 안 나와요
```

3 insertions(+), 1 deletion(-)은 diff의 + 줄 3개, - 줄 1개예요.



git show

git show는 커밋 하나가 바꾼 것을 보여 줘요

```
$ git show
commit 7e5303ce32924d91ff86f6f48d7f1d1af808f9ae (HEAD -> main)
Author: kim <kim@example.com>
Date:   Thu Oct 29 10:02:00 2026 +0900

    feat: 로그인 화면에 안내와 버튼 추가

diff --git a/login.md b/login.md
index ef88ce3..60a5971 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,5 @@
 # 로그인
 아이디
-비밀번호
+비밀번호 (8자 이상)
+로그인 버튼
+아이디 저장
```

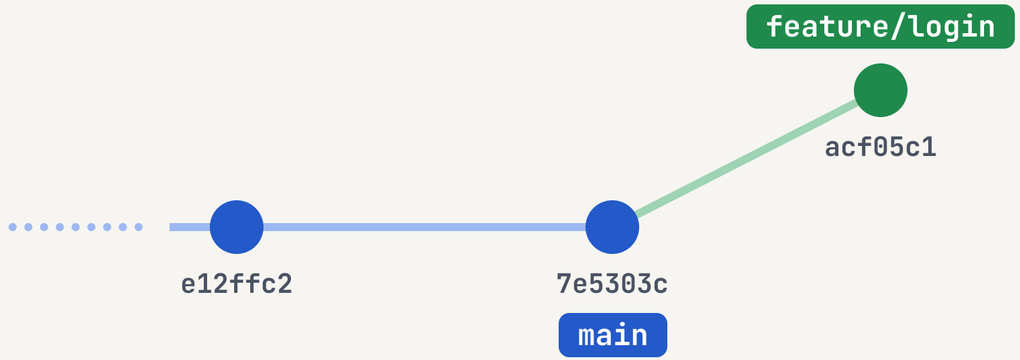
커밋 정보 번호, 만든 사람, 날짜, 메시지

바뀐 내용 바로 전 커밋과 비교한 diff

다른 커밋은 **`git log --oneline`**에서 번호를 찾아 **`git show e12ffc2`**처럼 봐요.

브랜치 비교

merge 전에 무엇이 들어올지 미리 봐요



앞에 쓴 쪽
main

→

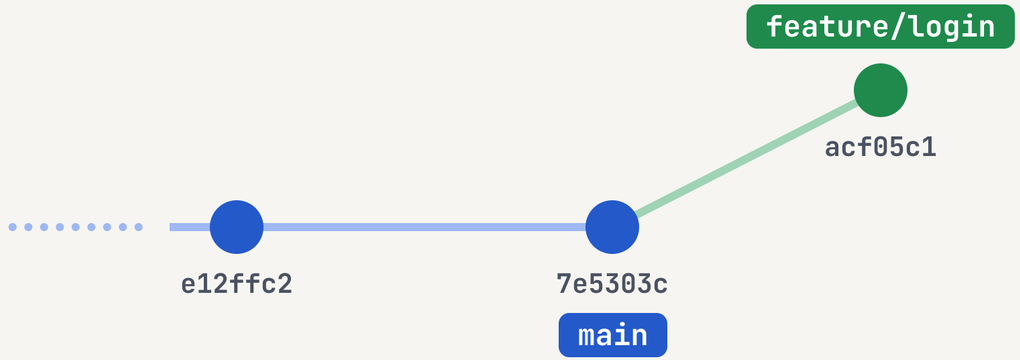
뒤에 쓴 쪽
feature/login

main을 feature/login으로 만들려면 무엇을 바꾸나

```
$ git diff main feature/login
diff --git a/login.md b/login.md
index 60a5971..14b3837 100644
--- a/login.md
+++ b/login.md
@@ -1,5 +1,6 @@
-# 로그인                    ← main에만 있는 줄
+# 로그인하기                ← feature/login에만 있는 줄
 아이디
 비밀번호 (8자 이상)
 로그인 버튼
 아이디 저장
+비밀번호 찾기              ← feature/login에서 새로 생긴 줄
```

브랜치 비교

순서를 바꾸면 +와 -가 뒤집혀요



앞에 쓴 쪽
feature/login

→

뒤에 쓴 쪽
main

feature/login을 main으로 만들려면 무엇을 바꾸나

merge 전에는 받을 쪽 main을 앞에 써요.

```
$ git diff feature/login main
diff --git a/login.md b/login.md
index 14b3837..60a5971 100644
--- a/login.md
+++ b/login.md
@@ -1,6 +1,5 @@
-# 로그인하기          ← feature/login에만 있는 줄
+# 로그인              ← main에만 있는 줄
 아이디
 비밀번호 (8자 이상)
 로그인 버튼
 아이디 저장
-비밀번호 찾기         ← 앞 장에서는 + 였던 줄
```

.gitignore

비밀 값이 든 .env는 올리면 안 돼요

<> ① 작업 폴더 hello-git/

login.md

커밋됨 · 바뀐 것 없음

.env

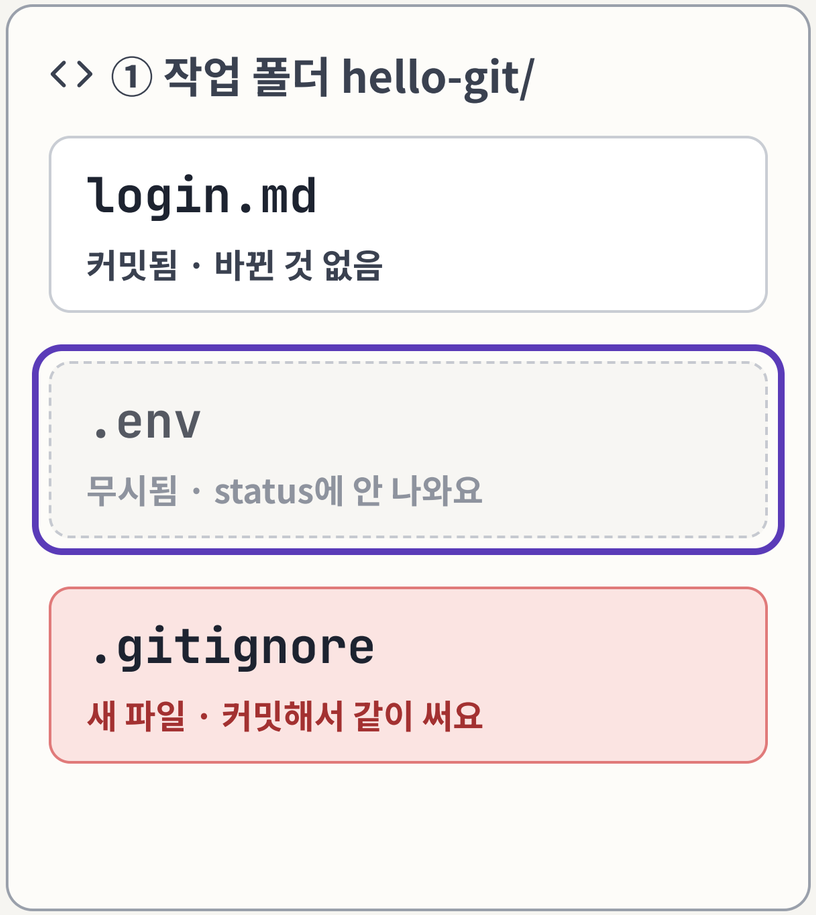
Untracked · API_KEY=abc123

```
$ echo "API_KEY=abc123" > .env
$ git status
On branch main
Untracked files:
        .env
```

git add . 한 번이면 .env까지 올라가요.
GitHub에 올라간 키는 누구나 볼 수 있어요.

.gitignore

.gitignore에 이름을 적으면 Git이 못 본 척해요




```
$ echo ".env" > .gitignore
$ git status
On branch main
Untracked files:
        .gitignore
$ git add .gitignore
$ git commit -m "chore: .env를 gitignore에 추가"
[main 0e35512] chore: .env를 gitignore에 추가
```

.env가 목록에서 빠졌어요

.env 파일은 그대로 있고 Git만 못 본 척해요.
.gitignore는 커밋해서 팀원도 같은 규칙을 써요.

.gitignore

자주 적는 이름은 정해져 있어요

`.env`	**비밀 값**	API 키, 비밀번호
`node_modules/`	**받은 패키지**	다시 받으면 돼요
`__pycache__/`	**파이썬 임시 파일**	실행하면 저절로 생겨요
`.venv/`	**파이썬 가상환경**	uv sync로 다시 만들어요
`*.log`	**로그 파일**	* 는 아무 이름
`.DS_Store`	**macOS 폴더 설정**	Finder가 만들어요

끝에 / 를 붙이면 폴더, * 는 아무 글자, # 으로 시작하면 메모 줄이에요.

.gitignore 파일 안

```
.env

node_modules/

__pycache__/

.venv/

*.log

.DS_Store
```

한눈에 비교

status를 보고 어떤 diff를 칠지 골라요

언제	status에 보이는 것	칠 명령	비교하는 두 곳
add 전	빨간 글씨 파일	`git diff`	① 작업 폴더 ↔ ② 대기실
commit 전	초록 글씨 파일	`git diff --staged`	② 대기실 ↔ ③ 마지막 커밋
commit 뒤	nothing to commit	`git show`	마지막 커밋 ↔ 바로 전 커밋
merge 전	브랜치가 두 개	`git diff main 브랜치`	두 브랜치의 마지막 커밋

빈 화면은 오류가 아니라 비교한 두 곳이 같다는 뜻이에요.

막힐 때

git diff에 아무것도 안 나와요

1 이미 git add 했어요

대기실에 올린 건 --staged로 봐요

```
git diff --staged
```

---

2 새로 만든 파일이에요

Untracked 파일은 git diff에 안 나와요. add한 뒤에 봐요

```
git add 파일
git diff --staged
```

---

3 이미 commit 했어요

커밋한 내용은 git show로 봐요

```
git show
```

셋 다 아니면 git status부터 봐요. 고친 파일이 없으면 diff도 비어요.

막힐 때

화면이 안 끝나면 q, 너무 길면 --stat

맨 아래에 : 나 (END)가 보이고 멈춰요

```
$ git show
commit 7e5303ce32924d91ff86f6f48d7f1d1af808f9ae (HEAD -> main)
Author: kim <kim@example.com>
Date:   Thu Oct 29 10:02:00 2026 +0900

    feat: 로그인 화면에 안내와 버튼 추가

diff --git a/login.md b/login.md
index ef88ce3..60a5971 100644
--- a/login.md
+++ b/login.md
@@ -1,3 +1,5 @@
 # 로그인
 아이디
:
```

여기서 멈춰 있어요

q

q를 누르면 나와요. Space는 다음 화면이에요.

파일 이름과 줄 수만 볼 때

```
$ git diff --stat main feature/login
 login.md | 3 ++-
 1 file changed, 2 insertions(+),
 1 deletion(-)
```

+, - 개수로 얼마나 바뀌었는지 봐요.
git diff, --staged, git show에 모두 붙여요.

함정

이미 커밋한 파일은 .gitignore로 숨겨지지 않아요

```
$ echo "*.log" >> .gitignore
$ echo "error" >> app.log
$ git status
Changes not staged for commit:
        modified:   .gitignore
        modified:   app.log
$ git rm --cached app.log
rm 'app.log'
$ git status
Changes to be committed:
        deleted:    app.log
Changes not staged for commit:
        modified:   .gitignore
$ git add .gitignore
$ git commit -m "chore: 로그 파일을 기록에서 빼기"
```

*.log를 적었는데 계속 나와요

기록에서만 빠져요

.gitignore는 Git이 아직 모르는 파일에만 통해요

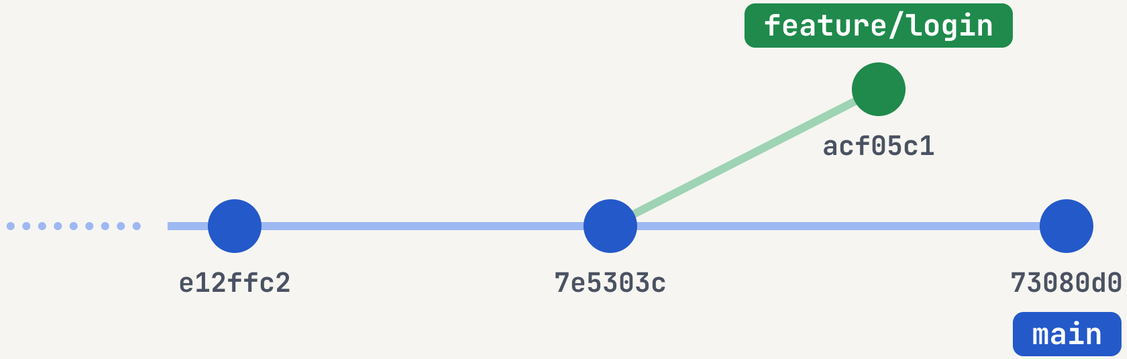
이미 커밋한 파일은 기록 대상에서 빼요.

`git rm --cached 파일`

내 폴더의 파일은 그대로 있어요.
그다음 .gitignore와 같이 add, commit 해요.

실제 상황 1 · merge 전에 충돌 미리 보기

같은 줄을 양쪽에서 고쳤는지 merge 전에 봐요



앞에 쓴 쪽
main

→

뒤에 쓴 쪽
feature/login

main을 feature/login으로 만들려면 무엇을 바꾸나

같은 줄이 양쪽에서 달라요. 이대로 merge하면 충돌이 나요.

```
$ git diff main feature/login
diff --git a/login.md b/login.md
index cea7b97..14b3837 100644
--- a/login.md
+++ b/login.md
@@ -1,5 +1,6 @@
-# 로그인 페이지          ← main의 제목
+# 로그인하기             ← feature/login의 제목
 아이디
 비밀번호 (8자 이상)
 로그인 버튼
 아이디 저장
+비밀번호 찾기
```

실제 상황 1 · merge 전에 충돌 미리 보기

충돌이 나면 git diff로 충돌 난 곳만 봐요

```
$ git merge feature/login
Auto-merging login.md
CONFLICT (content): Merge conflict in login.md
Automatic merge failed; fix conflicts and then commit the result.
$ git diff
diff --cc login.md
index cea7b97,14b3837..0000000
--- a/login.md
+++ b/login.md
@@@ -1,4 -1,4 +1,8 @@@
++<<<<<<< HEAD
 +# 로그인 페이지
++=======
+ # 로그인하기
++>>>>>>> feature/login
   아이디
   비밀번호 (8자 이상)
   로그인 버튼
```

++ 로 시작하는 줄이 충돌 표시

↑ 위: main(HEAD) 쪽

↓ 아래: feature/login 쪽

파일이 여러 개여도 git diff 하나로 충돌 표시가 남은 곳을 모두 찾아요.

실제 상황 1 · merge 전에 충돌 미리 보기

고친 뒤 add하고, 커밋 전에 --staged로 한 번 더 봐요

```
# login.md에서 # 로그인하기만 남기고 표시 세 줄을 지움
$ git add login.md
$ git diff --staged
diff --git a/login.md b/login.md
index cea7b97..14b3837 100644
--- a/login.md
+++ b/login.md
@@ -1,5 +1,6 @@
-# 로그인 페이지
+# 로그인하기
 아이디
 비밀번호 (8자 이상)
 로그인 버튼
 아이디 저장
+비밀번호 찾기
$ git commit -m "fix: 로그인 제목 충돌 해결"
[main c36ee83] fix: 로그인 제목 충돌 해결
```

--staged는 main의 마지막 커밋과 비교해요. 이 merge 커밋에 들어갈 내용이에요.

커밋 전에 볼 것

1. <<< 같은 표시가 남지 않았나
2. 남긴 제목이 맞나
3. main에 들어올 줄이 이게 다인가

실제 상황 2 · 커밋 전에 빠진 파일 찾기

커밋 전에 --stat으로 파일 목록부터 훑어요

```
$ echo "회원가입 링크" >> login.md
$ echo "# 회원가입" > signup.md
$ git add login.md
$ git diff --staged --stat
 login.md | 1 +
 1 file changed, 1 insertion(+)        ← 고친 건 두 파일인데 하나뿐이에요
$ git status
Changes to be committed:
        modified:   login.md
Untracked files:
        signup.md                     ← 회원가입 파일이 빠졌어요
$ git add signup.md
$ git diff --staged --stat
 login.md  | 1 +
 signup.md | 1 +                      ← 이제 두 파일이 다 들어가요
 2 files changed, 2 insertions(+)
$ git commit -m "feat: 회원가입 화면과 링크 추가"
[main ae7574c] feat: 회원가입 화면과 링크 추가
```

새 파일은 git diff에 안 나와서 놓치기 쉬워요. 커밋 전에 파일 개수를 세어 봐요.

한 장 요약

add 전	`git diff`	status 빨간 글씨 파일의 고친 줄
commit 전	`git diff --staged`	커밋에 들어갈 줄, --stat으로 파일 수
commit 뒤	`git show`	커밋 하나가 바꾼 것
merge 전	`git diff main 브랜치`	받을 쪽을 앞에, 충돌 나면 git diff
처음부터	`.gitignore`	.env, .venv/, __pycache__/, *.log

```
- 지운 줄    + 넣은 줄    @@ 위치    q 나가기
```

다음 05에서는 VS Code Git Graph로 이 기록을 그림으로 봐요.